

Assignment 4: Simulating a Converted SNN

Nitin Sai Maradani [EE23B141]

CS6886: Systems Engineering for Deep Learning

September 14, 2026

1. Task 1: Environment Setup and Baseline Evaluation

The assignment uses SpikingJelly, a PyTorch-based spiking neural network library, to simulate converted SNNs. After successful installation via `setup.py`, all imports executed smoothly and the pre-trained 3-layer CNN model was loaded.

The CNN architecture consists of three convolutional blocks:

- **Block structure:** Each block contains Conv2d, BatchNorm2d, ReLU, and AvgPool2d modules
- **Input:** 28×28 grayscale MNIST images (1 channel)
- **Conv layers:** $1 \rightarrow 32 \rightarrow 32 \rightarrow 32$ channels with 3×3 kernels
- **Pooling:** 2×2 average pooling with stride 2 after each block
- **Classification head:** Flatten + Linear(32, 10)
- **Baseline CNN Accuracy:** 98.70%

This ground-truth accuracy serves as the reference for evaluating energy-accuracy trade-offs in converted SNNs.

2. Task 2: MAC Operations Per Convolutional Layer

The number of MAC operations per convolutional layer:

$$\text{MACs} = C_{\text{in}} \times k_H \times k_W \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$$

where spatial output dimensions are computed as:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - k_H + 2p}{s} \right\rfloor + 1$$

The code snippet in the cell iterates through `model.network`, extracting kernel size, stride, padding, and channel counts from each Conv2d layer, while tracking spatial dimension changes through AvgPool2d layers.

Results (per test image):

Table 1: MAC Operations Per Convolutional Layer

Layer	Dimensions	Output Size	MACs
Conv0	$1 \rightarrow 32, 3 \times 3, 26 \times 26$	21,632 neurons	194,688
Conv1	$32 \rightarrow 32, 3 \times 3, 11 \times 11$	3,872 neurons	1,115,136
Conv2	$32 \rightarrow 32, 3 \times 3, 3 \times 3$	288 neurons	82,944
Total			1,392,768

3. Task 3: Comparison of Conversion Job Modes

ANN-to-SNN conversion replaces ReLU activations with $\text{VoltageScaler}(1/\lambda) \rightarrow \text{IFNode}(v_threshold=1.0) \rightarrow \text{VoltageScaler}(\lambda)$, where λ is the scaling factor. The mode parameter determines how λ is calibrated:

- **max**: $\lambda = \max(\text{activation})$ across calibration batches — highest sparsity, slowest convergence
- **99.9% max**: $\lambda = 99.9\text{th percentile}(\text{activation})$ — balanced sparsity and accuracy
- **1/2 max**: $\lambda = 0.5 \times \max(\text{activation})$ — moderate firing rates
- **1/4 max**: $\lambda = 0.25 \times \max(\text{activation})$ — fastest saturation, lowest sparsity

Observations across different conversion jobs:

- **1/4 max** reaches saturation fastest but exhibits high firing rates (poor sparsity)
- **max** achieves the highest sparsity but requires more timesteps to converge
- **99.9% max** provides the optimal trade-off: near-baseline accuracy with good sparsity

Design Choice: I selected **99.9% max** - since it was the Pareto optimal point between accuracy vs. sparsity.

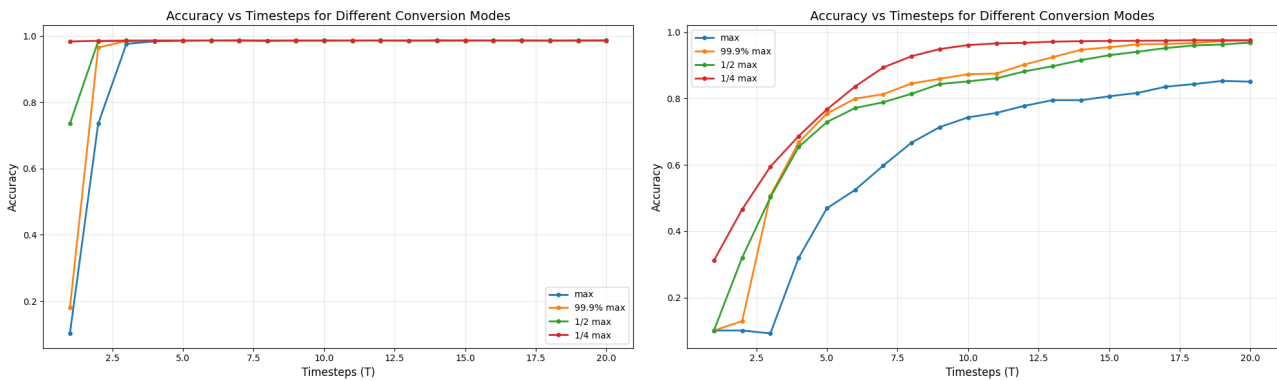


Figure 1: Conversion-job comparison (a) For 1 layer (b) all layers converted

4. Task 4: Extent of Conversion

To evaluate the impact of partial versus full conversion, I compared three scenarios:

- **1 layer converted** (`split_index = 8`): only Conv2 operates as SNN
- **2 layers converted** (`split_index = 4`): Conv1 and Conv2 as SNN
- **3 layers converted** (`split_index = 0`): full SNN conversion

The plot includes the baseline CNN accuracy (98.70%) as a horizontal reference line.

Fully-converted SNNs exhibit an initial accuracy drop but reach saturation given sufficient timesteps, demonstrating the feasibility of complete ANN-to-SNN conversion.

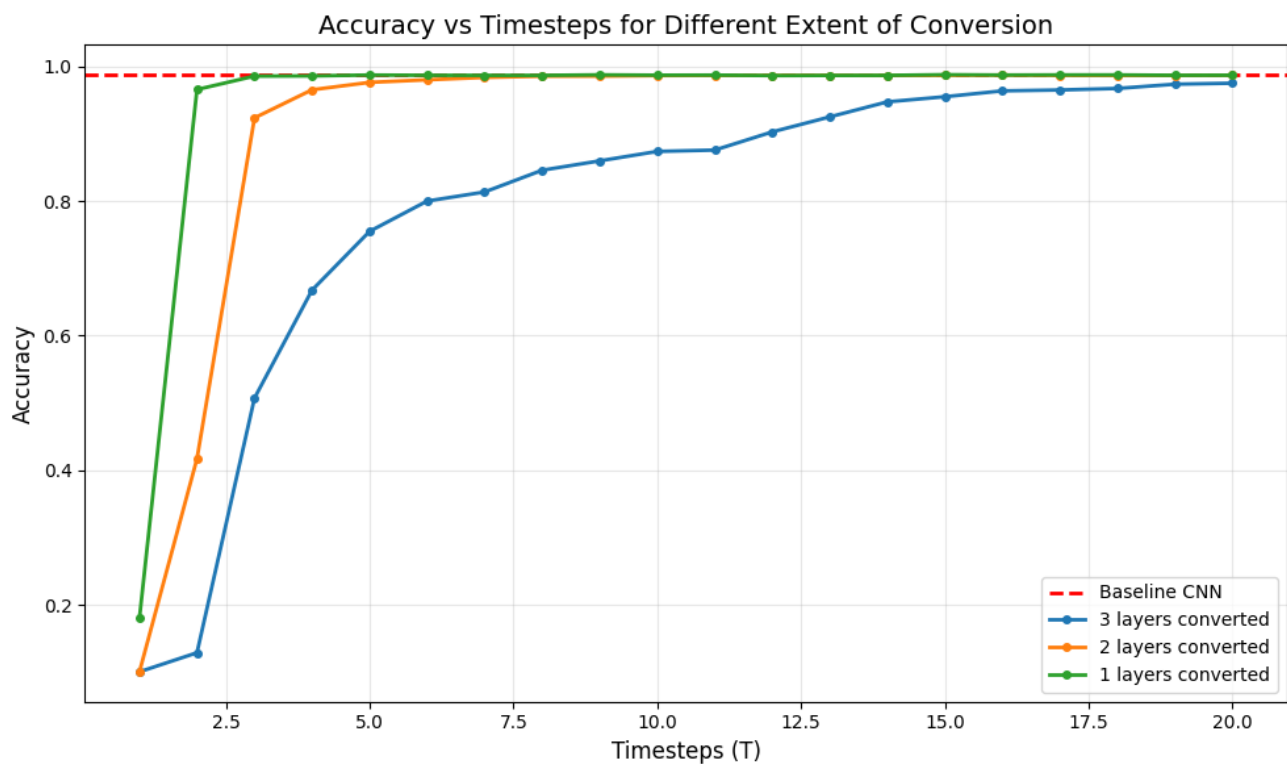


Figure 2: Accuracy vs. Timesteps with increasing extent of ANN-to-SNN conversion

5. Task 5: Layer-wise Spike Counts and Firing Rates

- Modified `val()` to create a `custom_val()` with 3 hooks, 1 for each layer (originally it had only 1, for the first layer).
- Spikes are normalized over the dataset length and time steps.
- The `spike_count` dict must be cleared after every iteration since it's a global variable being modified.

Results (99.9% max, $T = 20$, per test image):

Table 2: Spike Counts and Firing Rates

Layer	Neurons	Spike Count	Firing Rate
IFNode0	21,632	16,661.06	3.85%
IFNode1	3,872	5,493.08	7.09%
IFNode2	288	673.97	11.70%

Observation: Firing rates increase in deeper layers, reflecting higher per-neuron activity as spatial resolution decreases and feature abstraction intensifies.

6. Task 6: Best Accuracy for $\geq 90\%$ Energy Savings

Energy savings in SNNs arise from replacing dense MAC operations (4.6 nJ) with sparse event-driven Accumulate (AC) operations (0.6 nJ).

Assumption:

- Conv0 uses fixed MACs (4.6 nJ each) because its input is continuous FP32, not time-series spikes. Only Conv1 and Conv2 benefit from AC-based savings.
- Logic: For the compute to be AC, the input MUST BE from an IF Node layer; hence, even for full-conversion, the first Conv layer will be MAC.
- Following this logic, to achieve $\geq 90\%$ energy savings we have to use fully converted SNNs.

Accumulate (AC) Operation Count Formula:

$$ACs_{L+1} = \text{Firing_Rate}_L \times T \times MACs_{L+1}$$

I swept T from 20 down to 15, computing:

$$\begin{aligned} \text{SNN Energy} &= MACs_{\text{Conv0}} \times 4.6 + (ACs_{\text{Conv1}} + ACs_{\text{Conv2}}) \times 0.6 \\ \text{Ideal Savings} &= \frac{\text{ANN Energy (Conv1+Conv2)} - \text{SNN Energy (Conv1+Conv2)}}{\text{ANN Energy (Conv1+Conv2)}} \end{aligned}$$

Result: At $T = 19$:

- **Accuracy:** 97.33%
- **SNN Energy:** 1,443,895.43 nJ
- **ANN Energy:** 6,406,732.80 nJ
- **Ideal Savings (Conv1+Conv2):** 90.05%
- **Total Net Savings:** 77.46%

$T = 19$ is the minimum timestep configuration achieving $\geq 90\%$ energy savings on converted layers while maintaining high accuracy (97.33%).

7. Task 7: Best Energy Savings for $< 5\%$ Accuracy Loss

The constraint is: SNN accuracy $\geq 93.70\%$ (baseline 98.70% minus 5%).

I swept T from 15 down to 11:

Table 3: Accuracy and Energy Savings vs. Timesteps

T	Accuracy	Loss (%)	Ideal Savings	Constraint
15	95.47%	3.23%	92.58%	✓
14	94.71%	3.99%	93.26%	✓
13	92.48%	6.22%	93.95%	×

Result: At $T = 14$:

- **Accuracy:** 94.71% (loss = 3.99% $< 5\%$)
- **Ideal Savings (Conv1+Conv2):** 93.26%
- **Total Net Savings:** 80.22%

$T = 14$ maximizes energy savings while satisfying the accuracy degradation bound.

8. Task 8: Memory-Aware CNN Write Costs

For high-bandwidth memory with 25 nJ per 64-byte write:

Fused Block Output Sizes (after AvgPool):

- Fused0: $13 \times 13 \times 32 = 5,408$ activations
- Fused1: $5 \times 5 \times 32 = 800$ activations
- Fused2: $1 \times 1 \times 32 = 32$ activations

For FP32 (4 bytes/activation), 16 activations fit per 64-byte packet:

$$\text{Write Cost} = \left\lceil \frac{\text{Activation Count}}{16} \right\rceil \times 25 \text{ nJ}$$

Table 4: CNN Memory Write Costs

Block	Activations	Packets	Write Cost (nJ)
Fused0	5,408	338	8,450
Fused1	800	50	1,250
Fused2	32	2	50
Total			9,750

9. Task 9: Memory-Aware SNN Energy Model

SNNs emit binary spikes (1 bit/activation), packing 512 activations per 64-byte packet:

$$\text{SNN Write Cost per timestep} = \left\lceil \frac{\text{Activation Count}}{512} \right\rceil \times 25 \text{ nJ}$$

Table 5: SNN Memory Write Costs (per timestep)

Block	Activations	Packets	Write Cost (nJ)
Fused0	5,408	11	275
Fused1	800	2	50
Fused2	32	1	25
Total			350

At $T = 14$ (the optimal operating point from Task 7):

$$\text{CNN Total Energy} = 6,406,732.80 + 9,750 = 6,416,482.80 \text{ nJ}$$

$$\text{SNN Total Energy} = 1,267,093.59 + (350 \times 14) = 1,267,093.59 + 4,900 = 1,271,993.59 \text{ nJ}$$

$$\text{Net Savings} = \frac{6,416,482.80 - 1,271,993.59}{6,416,482.80} = 80.18\%$$

Ideal Savings (Conv1+Conv2 only): 93.42%